

Yes. After one more live verification, **I would build this project**. I would stop searching for a completely different idea unless you discover something fundamentally new. The concept is strong enough to compete for **Taskmaster and potentially the Grand Prize**, but only if we build the *RealityCI loop*, not merely a Gaussian simulator.

The current hackathon is explicitly asking for autonomous systems that make decisions, take actions, manage multi-step workflows, handle state/failures, and visibly prove execution. Taskmaster is the correct track. The judging weight is **40% Innovation & Operational Utility, 30% Architecture, 30% Demo/Production Readiness**.

## Final version of the idea

The product I would submit is:

**RealityCI — autonomous CI/CD for physical AI. Give it a physical-AI policy, a vehicle/robot and sensor configuration, and real-world recordings. It builds executable worlds, runs the model, discovers capability failures, determines why they happened, generates targeted training experiences, retrains through whatever training interface that model supports, independently checks generalization and regressions, and decides whether the new checkpoint is safe to promote. When it lacks the reality needed to fix a weakness, it autonomously identifies or requests the missing real-world experience.**

The loop is:

**Model → Drive → Failure → Causal Diagnosis → Experience Selection/Generation → Train → Hidden Exam → Regression Test → Accept/Reject Checkpoint → Reality Debt → Next Weakness → repeat.**

That is the idea. **Gaussian reconstruction is a subsystem, not the product.**

## Is it actually original enough?

This remains the biggest risk. Waymo already has a Genie-3-based world model that generates controllable camera and LiDAR driving simulations. NVIDIA NuRec already converts real sensor data into interactive Gaussian-based simulations. And Applied Intuition's brand-new Dana is explicitly an agentic physical-AI platform spanning development, simulation, testing and real-world data workflows.

So **do not pitch** "we automatically build simulations and train autonomous cars." That will sound derivative.

The defensible differentiation is the **capability-closing CI loop**: policy-agnostic integration, causal failure experiments, automatic selection of a training method, active acquisition of missing reality, hidden generalization testing, regression protection, Reality Debt, and automatic

checkpoint promotion/rejection. Based on the public materials I checked, I do not see Waymo, NuRec, or Dana publicly describing that exact general-purpose end-to-end workflow as a model-agnostic CI system. That's an inference from their published capabilities, not a claim that nobody internally has ever built it.

**My score:** concept around **9.2/10**. If the core demo is genuinely functional, I could see it producing a **92–96/100-quality submission**. That is *not* a 92–96% probability of winning; with hundreds of participants, nobody can credibly give you that probability. The technical execution will decide this. Devpost currently shows about 780 participants.

## The Google architecture I would lock in

Keep **Qt 6 + QML**. Don't change it. The desktop software remains the main user experience.

| Component                    | What we use it for                                                                             | Decision                 |
|------------------------------|------------------------------------------------------------------------------------------------|--------------------------|
| <b>Qt 6 + QML</b>            | Native RealityCI desktop UI                                                                    | <b>Keep</b>              |
| <b>Python + PyTorch</b>      | Agents, training, adapters, ML                                                                 | <b>Core</b>              |
| <b>C++/CUDA + Vulkan</b>     | Gaussian rendering / performance-critical runtime                                              | <b>Core where needed</b> |
| <b>Google ADK 2.0 Python</b> | Main agent graph: Scientist<br>→ Diagnostician → Trainer<br>→ Examiner → WorldScout            | <b>Primary framework</b> |
| <b>Google GenAI SDK</b>      | Direct Gemini multimodal calls, structured outputs, function calling, video/key-frame analysis | <b>Use</b>               |

|                              |                                                                   |                          |
|------------------------------|-------------------------------------------------------------------|--------------------------|
| <b>Antigravity SDK</b>       | Sandboxed Recovery Engineer that fixes broken adapters/code/tests | <b>Use if reliable</b>   |
| Genkit                       | Duplicates other layers                                           | <b>Skip</b>              |
| <b>Gemini 3.6 Flash</b>      | Main Scientist, causal diagnosis, difficult multimodal reasoning  | <b>Primary model</b>     |
| <b>Gemini 3.5 Flash-Lite</b> | Cheap event classification, telemetry summarization, subagents    | <b>High-volume model</b> |
| Gemini 3.5 Flash             | Independent fallback/evaluator if useful                          | <b>Optional</b>          |
| <b>Vertex AI</b>             | Run production Gemini calls under the Google Cloud deployment     | <b>Use</b>               |
| <b>Cloud Run</b>             | ADK/control API                                                   | <b>Use</b>               |
| <b>Firestore</b>             | Reality Debt, experiments, failures, checkpoints, workflow state  | <b>Use</b>               |
| <b>Pub/Sub</b>               | FAILURE_DETECTED,<br>TRAIN_REQUESTED,                             | <b>Use</b>               |

|                               |                                               |             |
|-------------------------------|-----------------------------------------------|-------------|
|                               | CHECKPOINT_READY,<br>EXAM_PASSED, etc.        |             |
| Cloud Storage                 | Videos, worlds,<br>checkpoints, run artifacts | Use         |
| Cloud SQL                     | No real need                                  | Skip        |
| GKE                           | Too much complexity for<br>this project       | Skip        |
| Cloud Run GPU / Nebius<br>GPU | Reconstruction/training<br>worker             | Either/both |
|                               |                                               |             |

ADK 2.0 is particularly appropriate because Google added graph workflows, concurrency controls, task-oriented agents and unified telemetry—the exact structure your long-running Scientist workflow needs. Google also explicitly documents durable, event-driven, long-running ADK agents rather than agents that constantly poll.

The GenAI SDK should **not duplicate ADK**. ADK is orchestration; GenAI SDK is how low-level services make direct Gemini calls. Google’s current Interactions API supports multimodal input, structured output and function calling, and Google recommends the GenAI SDK for current Gemini access.

Antigravity also has a legitimate role. Its current SDK can autonomously read files, execute commands and edit code. Make it a restricted **Recovery Engineer**, never the driving policy: if a customer’s sensor adapter crashes, Antigravity gets a temporary worktree, diagnoses it, changes code, runs tests and returns a candidate patch.

## Gemini 3.6 + 3.5

I would **not spend heavily on Gemini 3.5 Flash just because you want two model numbers**. Google now says 3.6 Flash is stronger on complex agentic/multimodal tasks *and*

*cheaper* than 3.5 Flash. Standard pricing is currently \$1.50/M input and \$7.50/M output for 3.6 Flash versus \$1.50/M and \$9/M for 3.5 Flash.

So use:

**Gemini 3.6 Flash → Scientist / Causal Analyst / difficult Examiner**

and

**Gemini 3.5 Flash-Lite → cheap triage / telemetry compression / event routing / schema extraction.**

3.5 Flash-Lite currently costs \$0.30/M input and \$2.50/M output at standard pricing, so using it for thousands of mundane subagent decisions makes much more sense.

If you specifically want **Gemini 3.5 Flash**, give it one real role: independent second-opinion failure adjudication. Don't route routine calls through it.

## **We should actually use more Google AI — for the bonus**

This hackathon has unusually valuable explicit bonus points: **+0.2 for a public build article/video, +0.2 for the qualifying social post, and +0.2 per additional Google AI model up to +0.6**. The final score can reach 6 instead of the normal 5.

## **Google Cloud's RTX PRO 6000 changes my GPU recommendation**

This surprised me during the final check.

Cloud Run now supports the **RTX PRO 6000 Blackwell with 96 GB VRAM**, including Cloud Run Jobs. It can scale to zero and Google currently lists the raw GPU portion at \$0.00036522/sec—about **\$1.31/hour for the GPU itself**, before the required CPU/RAM and other charges. It does require at least 20 vCPUs and 80 GiB host RAM, and availability is currently region-limited.

So I would test both:

**Google Cloud Run Job / RTX PRO 6000** for the final Google-native demonstration, and **Nebius** as your cheaper/flexible development or fallback compute.

This is actually better for the story. Imagine Gemini decides to retrain and the UI shows:

**TRAINING\_REQUIRED**

ADK → Pub/Sub → Cloud Run GPU Job

RTX PRO 6000 allocated  
checkpoint v18 created  
Examiner launched.

That's extremely strong **Proof of Action** for the hackathon.

One caveat: Cloud Run is fundamentally container-based. You don't need Docker throughout the desktop project, and you can deploy ordinary Cloud Run services from source, but a CUDA-heavy GPU training job will realistically be packaged as a container. Keep containerization isolated to `cloud/gpu_worker/`; don't restructure the whole project around Docker.

## GPT/Nebius during development are fine

The hackathon's FAQ explicitly says AI assistants are welcome for scaffolding, debugging and iteration. So using your existing GPT API budget to write/debug the software does not conflict with the contest. The submitted product still needs to satisfy its actual Gemini + Google agent framework + Google Cloud requirements.

Similarly, third-party integrations are permitted as long as you're authorized to use them and comply with their licenses/terms. You can have external GPU workers while Google Cloud remains the agent/control plane.

For the **video**, though, I would emphasize the Google path rather than showing giant Nebius/NVIDIA logos. The judges explicitly require visible proof that the backend ran on Google Cloud.

## What I would actually implement before adding more features

The winning demo does **not** require implementing every robot/model/trainer.

Architect for all of them, but make the real demo support **two policy adapters**: one reasonably small PyTorch policy that can genuinely retrain quickly, and one substantially different policy adapter proving RealityCI is not hardcoded to the first one. Alpamayo can be an optional adapter, not the center.

Then demonstrate one complete event:

**Car drives → rare scenario → crash → Gemini investigates → runs counterfactual experiments → identifies capability gap → ADK launches targeted training → weights actually change → new checkpoint → completely hidden world → car succeeds →**

**regression suite passes → checkpoint promoted → Reality Debt changes from ❌ to ✅  
→ agent automatically selects the next weakness.**

If you have that working, **do not add five more half-functional simulator integrations.**

After that works, add WorldScout missing-reality acquisition.

Then add Antigravity self-repair.

That ordering maximizes the judging rubric.

## **One thing that could kill the project**

Don't claim:

"RealityCI can train any model."

Say:

**"RealityCI can execute and evaluate arbitrary policies through adapters. If the policy exposes a supported training interface, RealityCI can autonomously train it. For inference-only or proprietary black-box models, it produces the diagnosed failure set and targeted training dataset instead."**

That is credible engineering.

Also, for the submitted demo, use footage you created or material you are clearly licensed to use. The contest rules explicitly require the submission not to violate third-party copyright/privacy rights and require you to comply with licenses for third-party tools/data.